WEEK 9 — Unit 3: Microservices with Docker Compose

Topic: Secrets, Configs, Service Dependency Ordering & Use Case Deployments Course: INT 332 – DevOps

---

Note 1: Secrets and Configs in Docker Compose

Why Secrets and Configs Exist

In Week 8, we saw that environment variables can be used to pass sensitive data like passwords into containers. However, environment variables have a key weakness — they are visible in plain text in the docker-compose.yml file, in process listings (ps aux), and in docker inspect output. For production environments, this is a serious security risk.

Docker introduced **secrets** and **configs** as more secure alternatives for managing sensitive data and configuration files.

Docker Secrets

What is a Secret?

A Docker secret is a blob of sensitive data — such as a password, an API key, a TLS certificate, or an SSH private key — that is stored securely and made available to containers that explicitly need it. Secrets are stored in Docker's internal store and are mounted into the container as a file, not as an environment variable.

How Secrets Work in Docker Compose

Secrets are mounted inside the container at /run/secrets/<secret-name>. The application must be written to read its credentials from a file at that path, rather than from an environment variable.

Defining Secrets in docker-compose.yml

Method 1: File-based secrets (used in development)

```
services:
  db:
    image: postgres:15
    secrets:
      - db_password
    environment:
```

```
    POSTGRES_PASSWORD_FILE: /run/secrets/db_password


secrets:
  db_password:
    file: ./secrets/db_password.txt
```

Here, ./secrets/db_password.txt is a plain text file on your host machine containing just the password. Docker reads this file and makes it available inside the container at /run/secrets/db_password.

Method 2: External secrets (used in Docker Swarm production mode)

```
secrets:
  db_password:
    external: true
```

This tells Compose that the secret was already created using docker secret create and should be referenced as-is.

Important Notes on Secrets

- Secrets are only available to services that explicitly list them under the secrets key
- The secret file inside the container is read-only
- In Docker Compose (non-Swarm), secrets are simulated using bind mounts — true secret encryption only happens in Docker Swarm mode
- Always add the secrets directory to .gitignore

Docker Configs

What is a Config?

Configs are similar to secrets but are intended for non-sensitive configuration data — for example, an Nginx configuration file, a custom nginx.conf, or an application settings file. Like secrets, configs are mounted as files inside the container.

The key difference is:

- **Secrets** — for sensitive data (passwords, keys); stored encrypted in Swarm
- **Configs** — for non-sensitive configuration files; stored unencrypted

Defining Configs in docker-compose.yml

```
services:
  web:
    image: nginx:latest
    configs:
      - source: nginx_config
        target: /etc/nginx/nginx.conf

configs:
  nginx_config:
    file: ./config/nginx.conf
```

Here, the nginx.conf file from your host is mounted into the container at /etc/nginx/nginx.conf. This is cleaner than using a bind mount because the config is managed through Docker's config system and can be versioned separately.

Config Options

```
configs:
  nginx_config:
    file: ./config/nginx.conf

# OR reference an externally created config:
configs:
  nginx_config:
    external: true
```

Secrets vs Environment Variables vs Configs — Summary

Feature	Environment Variables	Secrets	Configs
Visible in inspect	Yes	No	No
Suitable for passwords	Risky	Yes	No
Suitable for config files	No	No	Yes

Feature	Environment Variables	Secrets	Configs
Mounted as file	No	Yes (/run/secrets/)	Yes (custom path)
Encrypted at rest	No	Yes (Swarm only)	No
